

CSA0504 — Database Management Systems

Special Assessment 2

National Vocational Certification and Exam Slot-Booking System: An Integrated Study in File Organization, Indexing, Hashing, Query Optimization, and Transaction Management

An Integrated Study in File Organization, Indexing, Hashing, Query Optimization, and Transaction Management

Team Members

Kasula Hemanth

D. Devendra

P Shashank

Course Outcomes: CO4, CO5 | Bloom's Levels: L3–Apply, L4–Analyze

SDG Mapping: SDG 4 (Quality Education), SDG 8 (Decent Work), SDG 9 (Industry, Innovation & Infrastructure)

1. Problem Statement

The National Skill Certification Council conducts computer-based vocational certification exams at thousands of centers nationwide for millions of candidates every year. Every attempt generates detailed records — question-wise responses, timing data, and scores — which are used later for result computation, appeals, and analytics. Candidates also book their preferred exam date, time slot, and center in real time through an online portal, and once exams conclude, examiners and coordinators must process and publish results under strict deadlines.

Two distinct problems currently affect the Council's system:

- Slow retrieval from the attempt archive: The exam-attempt archive is stored as one continuously growing sequential file per year. A query such as "retrieve candidate X's full attempt history" or "list all attempts at center Y on a given date" requires a full scan of millions of records, which is far too slow for time-bound audits and appeals.
- Uncoordinated concurrent access: Slot booking and result processing are implemented as uncoordinated read-then-write operations. During high-demand booking windows, the same seat/time-slot at a center has been booked by two candidates at once (a lost-update / double-booking anomaly), and simultaneous result-processing updates by different examiners have occasionally overwritten one another, producing incorrect or inconsistent published scores.

The assignment requires designing a complete database solution that (a) reorganizes the attempt archive using an appropriate file organization, indexing, and hashing strategy, and (b) redesigns slot booking and result publishing as properly bounded, isolation-controlled transactions.

2. Objective

- To design and justify an efficient file organization (sequential / indexed-sequential / direct-hashed) for the high-volume exam-attempt archive.
- To design single-level/multilevel and B+-tree indexes that support fast candidate-wise point lookup and center-wise range/audit retrieval, distinguishing clustering from non-clustering indexes.

- To design a hashing scheme with a defined collision-resolution policy for near-constant-time candidate/attempt lookup.
- To design a normalized relational schema and optimized SQL queries (joins, subqueries, aggregates) for slot booking and result reporting.
- To analyze query execution plans, identify costly operations, and apply indexing/query-rewriting optimizations.
- To design ACID-compliant transactions for slot booking and result publishing using COMMIT, ROLLBACK, and SAVEPOINT, with an isolation/locking strategy that eliminates double-booking and lost updates.
- To benchmark and validate the proposed design against the existing flat-file, uncoordinated approach.

3. Requirements and Environment Used

3.1 Software / Tools

- RDBMS: MySQL 8.0 (InnoDB storage engine) — chosen for its support of B-tree indexes, hash indexing on MEMORY tables, row-level locking, and configurable transaction isolation levels. (Substitute PostgreSQL/pgAdmin or Oracle/SQL Developer if that is what your lab uses.)
- MySQL Workbench / pgAdmin / SQL Developer — schema design, query execution, and EXPLAIN plan visualization.
- draw.io / Lucidchart — ER diagram, index-structure, and transaction-flow diagrams.
- Two or more concurrent SQL client sessions (or a small Python/Java script using threads) to simulate simultaneous booking and result-update requests.

3.2 Hardware

Standard laboratory computer/personal computer — Intel/AMD multi-core processor, 8 GB+ RAM, running Windows/Linux, sufficient to host a local MySQL instance and simulate concurrent client sessions. (Replace with your actual machine specifications.)

3.3 Key Requirements Addressed

Category	Requirement
Functional	Store per-candidate, per-attempt records; fast point lookup by candidate; fast center-wise range retrieval; slot booking bounded by capacity; concurrent, conflict-free result publishing.
Performance	Avoid full-file scans for candidate-history/center-audit queries; efficient execution during high-demand booking and result-declaration windows.
Technical	Relational DBMS with index/hash support and configurable-isolation transactions; booking/publishing implemented as bounded transactions, not isolated statements.
Reliability / Integrity	Index/hash structures stay consistent after inserts/updates; a slot is never booked beyond capacity; a published result is never lost or partially overwritten.
Security	Access to responses, scores, and personal data restricted via GRANT/REVOKE privileges.
Scalability	Design accommodates growth in candidates, centers, and exam cycles without major redesign.

4. Design / Proposed Solution

4.1 File Organization for the Exam-Attempt Archive

The archive is redesigned from a single growing sequential file into an indexed-sequential file, physically clustered (sorted) on candidate_id within each yearly partition. This preserves the low storage overhead and efficient

sequential/range access of a sequential file, while a sparse primary index on candidate_id turns point lookups from an O(n) scan into an O(log n) index probe followed by a short sequential scan of the candidate's own attempts (typically 1–5 records, since each candidate attempts far fewer exams than the archive holds in total).

Because the single most frequent access pattern is "fetch this exact candidate's attempt record," a static hash file is layered on top for that pattern: candidate_id is hashed to a bucket that stores direct pointers (or the row itself, for a MEMORY/HASH-engine lookup table) to the candidate's attempt rows, giving near O(1) average-case retrieval instead of even the O(log n) index probe. This combination — indexed-sequential primary storage plus a hash-based fast path for point lookups — is justified below in 4.4.

4.2 Indexing Design

- Primary (clustering) index: B+-tree on attempts(candidate_id, attempt_date) — clustering because the underlying InnoDB table is physically ordered by this key, giving efficient range scans for "all attempts by candidate X."
- Secondary (non-clustering) index: B+-tree on attempts(center_id, exam_date) — supports the center-wise audit query ("all attempts at center Y on date Z") without touching the clustering key; the index leaf stores pointers back to the clustered rows.
- Multilevel structure: both indexes are implemented as B+-trees (InnoDB's native index structure), giving logarithmic search, ordered range scans, and efficient insert/delete rebalancing — preferable to a flat single-level index once the archive grows to millions of rows.

4.3 Hashing Scheme

A static hash index is defined (in MySQL, via a MEMORY-engine lookup table with USING HASH, or conceptually as a separate hash file) on candidate_id for the point-lookup pattern:

- Hash function: $h(\text{candidate_id}) = \text{candidate_id} \bmod N$, where N is a prime bucket count sized to the expected candidate population (chosen to keep average chain length short, e.g., load factor ≈ 0.7).
- Collision resolution: separate chaining — each bucket holds a linked list (implemented as a row group keyed by bucket_id) of colliding candidate_id entries, each pointing to that candidate's attempt records in the indexed-sequential store.
- Growth strategy: because the candidate population grows over time, the design notes dynamic (extendible) hashing as the production-scale alternative to avoid full rehashing; the static scheme is used here for simplicity of implementation and benchmarking within the assignment's scope.

4.4 Justification: Indexed-Sequential + Hashing vs. Alternatives

Criterion	Sequential (baseline)	Indexed-Sequential + Hash (proposed)	Pure Direct/Hashed only
Point lookup (candidate)	O(n) full scan	O(1) average via hash	O(1) average via hash
Range / center-date audit	O(n) full scan	O(log n) via B+-tree secondary index	Poor — hashing destroys ordering
Insert overhead	Low	Moderate (index + hash maintenance)	Moderate
Storage overhead	Lowest	Extra index/hash structures	Extra hash structure, no range support

The proposed hybrid was selected because the workload has two distinct patterns — exact-candidate lookup and ordered center/date audits — that a single access method cannot serve optimally alone. Pure hashing would optimize the first at the expense of the second, which auditors depend on.

4.5 Relational Schema for Booking and Result Processing

The booking/result subsystem is modeled as a normalized (3NF) relational schema:

- centers(center_id PK, name, address, total_capacity)
- slots(slot_id PK, center_id FK, exam_date, start_time, capacity, seats_booked, version)
- candidates(candidate_id PK, name_hash, contact_hash, registration_date) — personal fields access-restricted per 3.3
- bookings(booking_id PK, candidate_id FK, slot_id FK, booking_time, status, UNIQUE(candidate_id, slot_id))
- attempts(attempt_id PK, candidate_id FK, center_id FK, exam_date, responses_json, timing_seconds, raw_score)
- results(result_id PK, attempt_id FK, examiner_id FK, final_score, status, last_updated, version)

The seats_booked/capacity pair on slots and the version column on slots and results are the two integrity mechanisms that the transaction design in Section 4.6 relies on to prevent, respectively, over-booking and lost result updates.

4.6 Concurrency-Control / Transaction Strategy

- Slot booking → pessimistic locking: a booking transaction runs at REPEATABLE READ (MySQL/InnoDB default) and issues SELECT ... FOR UPDATE on the target slot row before checking seats_booked < capacity. This places an exclusive row lock, so a second concurrent booking for the same slot blocks until the first transaction commits or rolls back, which is what makes the capacity check safe.
- Result publishing → optimistic concurrency: each results row carries a version counter. An examiner reads the row (including its version), computes the update, and commits with UPDATE ... WHERE result_id = ? AND version = ?. If another examiner committed first, zero rows are affected, the transaction detects the conflict, rolls back, and the client re-reads and retries — avoiding long-held locks during result review while still preventing a lost update.
- Isolation level: REPEATABLE READ is used as the default for both flows; the booking flow additionally uses explicit row locking (FOR UPDATE) rather than relying on SERIALIZABLE, which would reduce throughput under heavy simultaneous booking load.

5. Algorithm / Pseudocode

5.1 Hashed Point Lookup (with Chaining)

```
FUNCTION lookup_candidate(candidate_id):
    bucket_id = candidate_id MOD N           // N = number of hash buckets
    chain = hash_table[bucket_id]           // linked list of entries
    FOR EACH entry IN chain:
        IF entry.candidate_id == candidate_id:
            RETURN fetch_attempts(entry.pointer) // pointer into indexed-sequential store
    RETURN NOT_FOUND

FUNCTION insert_candidate(candidate_id, attempt_pointer):
    bucket_id = candidate_id MOD N
    APPEND (candidate_id, attempt_pointer) TO hash_table[bucket_id] // collision -> chain grows
```

5.2 Center-Wise Audit Retrieval (B+-Tree Range Scan)

```
FUNCTION center_audit(center_id, exam_date):
    node = root_of(secondary_index_on_center_date)
    WHILE node is not a leaf:
        node = child_pointer_for_key(node, (center_id, exam_date))
    result_list = []
    WHILE leaf.key == (center_id, exam_date): // scan matching leaf entries
        result_list.APPEND(dereference(leaf.record_pointer))
        leaf = leaf.next_leaf_via_sibling_pointer
    RETURN result_list
```

5.3 Slot Booking Transaction (Pessimistic Locking)

```
PROCEDURE book_slot(candidate_id, slot_id):
    BEGIN TRANSACTION
    SAVEPOINT before_check
    SELECT capacity, seats_booked INTO cap, booked
    FROM slots WHERE slot_id = slot_id FOR UPDATE // exclusive lock, blocks rivals
    IF booked >= cap THEN
        ROLLBACK TO before_check
        RETURN REJECTED ("slot full")
    END IF
    INSERT INTO bookings(candidate_id, slot_id, status) VALUES (candidate_id, slot_id, 'CONFIRMED')
    UPDATE slots SET seats_booked = seats_booked + 1 WHERE slot_id = slot_id
    COMMIT
    RETURN CONFIRMED
```

5.4 Result Publishing Transaction (Optimistic Concurrency)

```
PROCEDURE publish_result(attempt_id, examiner_score):
    BEGIN TRANSACTION
    SELECT result_id, version INTO rid, ver FROM results WHERE attempt_id = attempt_id
    rows_affected = UPDATE results
        SET final_score = examiner_score, status = 'PUBLISHED',
        version = version + 1, last_updated = NOW()
        WHERE result_id = rid AND version = ver
    IF rows_affected == 0 THEN
        ROLLBACK
        RETURN CONFLICT ("result changed by another examiner – reload and retry")
    END IF
    COMMIT
    RETURN SUCCESS
```

6. Implementation / Source Code

The following MySQL/InnoDB DDL and DML implement the design in Section 4. Adjust names, engine choices, and paths to match the DBMS actually used in the lab.

6.1 Schema Creation

```
CREATE DATABASE exam_council;
USE exam_council;
```

```
CREATE TABLE centers (
  center_id      INT PRIMARY KEY AUTO_INCREMENT,
  name           VARCHAR(100) NOT NULL,
  address        VARCHAR(200),
  total_capacity INT NOT NULL
) ENGINE = InnoDB;
```

```
CREATE TABLE slots (
  slot_id        INT PRIMARY KEY AUTO_INCREMENT,
  center_id      INT NOT NULL,
  exam_date      DATE NOT NULL,
  start_time     TIME NOT NULL,
  capacity       INT NOT NULL,
  seats_booked   INT NOT NULL DEFAULT 0,
  version        INT NOT NULL DEFAULT 0,
  FOREIGN KEY (center_id) REFERENCES centers(center_id),
  CHECK (seats_booked <= capacity)
) ENGINE = InnoDB;
```

```
CREATE TABLE candidates (
  candidate_id    INT PRIMARY KEY AUTO_INCREMENT,
  name_hash       CHAR(64) NOT NULL,          -- store hashed/tokenised PII only
  contact_hash    CHAR(64),
  registration_date DATE NOT NULL
) ENGINE = InnoDB;
```

```
CREATE TABLE bookings (
  booking_id      INT PRIMARY KEY AUTO_INCREMENT,
  candidate_id    INT NOT NULL,
  slot_id         INT NOT NULL,
  booking_time    DATETIME DEFAULT CURRENT_TIMESTAMP,
  status          ENUM('CONFIRMED','CANCELLED') DEFAULT 'CONFIRMED',
  FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
  FOREIGN KEY (slot_id) REFERENCES slots(slot_id),
  UNIQUE (candidate_id, slot_id)
) ENGINE = InnoDB;
```

```
CREATE TABLE attempts (
  attempt_id      BIGINT PRIMARY KEY AUTO_INCREMENT,
  candidate_id    INT NOT NULL,
  center_id       INT NOT NULL,
  exam_date       DATE NOT NULL,
  responses_json  JSON,
  timing_seconds  INT,
  raw_score       DECIMAL(6,2),
  FOREIGN KEY (candidate_id) REFERENCES candidates(candidate_id),
```

```

    FOREIGN KEY (center_id) REFERENCES centers(center_id)
) ENGINE = InnoDB;

CREATE TABLE results (
    result_id      BIGINT PRIMARY KEY AUTO_INCREMENT,
    attempt_id     BIGINT NOT NULL UNIQUE,
    examiner_id    INT,
    final_score     DECIMAL(6,2),
    status          ENUM('PENDING','PUBLISHED') DEFAULT 'PENDING',
    last_updated    DATETIME,
    version         INT NOT NULL DEFAULT 0,
    FOREIGN KEY (attempt_id) REFERENCES attempts(attempt_id)
) ENGINE = InnoDB;

```

6.2 Indexes (B+-Tree, Clustering and Non-Clustering)

```

-- Clustering index: InnoDB automatically clusters attempts on its PRIMARY KEY.
-- To cluster physically on candidate_id instead, candidate_id is made a leading
-- column of the primary key in production; for this assignment a covering
-- secondary index approximates the clustering benefit:
CREATE INDEX idx_attempts_candidate ON attempts(candidate_id, exam_date);

-- Non-clustering secondary index for center-wise audit queries:
CREATE INDEX idx_attempts_center_date ON attempts(center_id, exam_date);

-- Supporting index for booking capacity checks:
CREATE INDEX idx_slots_center_date ON slots(center_id, exam_date);

```

6.3 Hash Index for Candidate Point Lookup

```

-- MySQL's MEMORY engine supports explicit HASH indexes; used here as a fast
-- lookup layer (bucket -> pointer) sitting in front of the InnoDB archive.
CREATE TABLE candidate_hash_lookup (
    candidate_id  INT NOT NULL,
    attempt_id    BIGINT NOT NULL,
    INDEX USING HASH (candidate_id)
) ENGINE = MEMORY;

-- Populate (kept in sync via trigger or application logic on insert):
INSERT INTO candidate_hash_lookup (candidate_id, attempt_id)
SELECT candidate_id, attempt_id FROM attempts;

-- Point lookup for a candidate's attempts (near O(1) average):
SELECT a.*
FROM candidate_hash_lookup h
JOIN attempts a ON a.attempt_id = h.attempt_id
WHERE h.candidate_id = 100234;

```

6.4 Candidate-History and Center-Audit Queries

```

-- Candidate attempt history (uses idx_attempts_candidate)
SELECT attempt_id, exam_date, raw_score, timing_seconds
FROM attempts
WHERE candidate_id = 100234
ORDER BY exam_date;

-- Center-wise audit for a given date (uses idx_attempts_center_date)
SELECT a.attempt_id, a.candidate_id, a.raw_score, r.final_score, r.status
FROM attempts a

```

```

LEFT JOIN results r ON r.attempt_id = a.attempt_id
WHERE a.center_id = 45 AND a.exam_date = '2026-08-15'
ORDER BY a.attempt_id;

```

6.5 Slot Booking Transaction (SQL)

```

START TRANSACTION;
SAVEPOINT before_check;

SELECT capacity, seats_booked
FROM slots
WHERE slot_id = 501
FOR UPDATE;                                -- exclusive row lock

-- Application checks: IF seats_booked >= capacity THEN ROLLBACK TO before_check; STOP;

INSERT INTO bookings (candidate_id, slot_id, status)
VALUES (100234, 501, 'CONFIRMED');

UPDATE slots
SET seats_booked = seats_booked + 1
WHERE slot_id = 501;

COMMIT;

```

6.6 Result Publishing Transaction (Optimistic Concurrency, SQL)

```

START TRANSACTION;

SELECT result_id, version
FROM results
WHERE attempt_id = 900321;                -- read current version, e.g. version = 3

UPDATE results
SET final_score = 78.5,
    status = 'PUBLISHED',
    version = version + 1,
    last_updated = NOW()
WHERE result_id = 1201 AND version = 3;    -- fails (0 rows) if another examiner already
updated it

-- Application checks ROW_COUNT(): if 0, ROLLBACK and re-read/retry; else COMMIT;
COMMIT;

```

6.7 Query Optimization Example (Before / After)

```

-- BEFORE (no index on center_id/exam_date): forces a full table scan
EXPLAIN SELECT * FROM attempts WHERE center_id = 45 AND exam_date = '2026-08-15';
-- type: ALL, rows: ~4,800,000 (full scan of the archive)

-- AFTER creating idx_attempts_center_date:
EXPLAIN SELECT * FROM attempts WHERE center_id = 45 AND exam_date = '2026-08-15';
-- type: ref, key: idx_attempts_center_date, rows: ~180 (index range scan)

```

6.8 Security: Access Control

```

CREATE USER 'examiner'@'%' IDENTIFIED BY 'strong_password';
GRANT SELECT, UPDATE ON exam_council.results TO 'examiner'@'%';
REVOKE DELETE ON exam_council.results FROM 'examiner'@'%';

```



```
CREATE USER 'auditor'@'%' IDENTIFIED BY 'strong_password';
GRANT SELECT ON exam_council.attempts TO 'auditor'@'%;
GRANT SELECT ON exam_council.results TO 'auditor'@'%';
```

7. Test Cases and Expected / Actual Results

#	Test Case	Expected Result	Actual Result	Status
1	Lookup a candidate's attempt record by candidate_id	Single matching record returned quickly	Record returned via hash lookup	Pass
2	Center-wise audit query for a given date	All matching attempt records listed	Correct records listed via index scan	Pass
3	Two candidates book the last available slot simultaneously	Only one booking succeeds; slot count never exceeds capacity	One booking accepted, other rejected consistently	Pass
4	Two examiners update the same candidate's result concurrently	Both updates applied without one overwriting the other	Second writer detects version conflict, retries, applies correctly	Pass
5	Rollback a booking transaction mid-way (simulated failure)	No partial booking data committed	Transaction fully rolled back	Pass
6	Compare optimized vs. unoptimized result-report query	Optimized query executes faster with fewer rows scanned	Reduced execution time observed after indexing	Pass

Note: replace "Actual Result" and "Status" with the outcomes you observe when you run these tests on your own dataset; record the exact execution times and row counts from your EXPLAIN/benchmark output in Section 9.

8. Execution Screenshots / Output

Insert screenshots captured from your own MySQL Workbench / pgAdmin / SQL Developer session below each corresponding item. Suggested evidence to capture:

- Table, index, and hash-structure creation (Section 6.1–6.3) — successful execution output.
- Sample data loaded into attempts, slots, bookings, and results.
- EXPLAIN plan output before and after adding idx_attempts_center_date (Section 6.7), showing the change from type: ALL to type: ref/range.
- Two concurrent sessions attempting to book the same slot — one COMMIT succeeding, one rejected/blocked.
- Two concurrent sessions updating the same result row — the version check causing the second UPDATE to affect 0 rows.
- A COMMIT, a ROLLBACK, and a SAVEPOINT-based partial rollback, each shown in the transaction log/console.
- Timing output (e.g., SHOW PROFILES, or wall-clock timing from your client) comparing baseline vs. optimized query execution.

[Insert Screenshot 1: Schema and index creation]

[Insert Screenshot 2: EXPLAIN plan — before optimization]

[Insert Screenshot 3: EXPLAIN plan — after optimization]

[Insert Screenshot 4: Concurrent booking test — one accepted, one rejected]

[Insert Screenshot 5: Concurrent result update — version-conflict retry]

[Insert Screenshot 6: Transaction log — COMMIT / ROLLBACK / SAVEPOINT]

9. Analysis and Discussion

9.1 Storage and Retrieval Efficiency

The flat sequential archive requires an $O(n)$ scan for both candidate lookup and center-wise audit, which becomes impractical once the archive reaches millions of rows. The proposed indexed-sequential organization with a B+-tree secondary index reduces the center-audit query to an $O(\log n)$ index probe plus a short leaf scan, and the hash lookup layer reduces the candidate point-lookup to near $O(1)$ average time. The trade-off is additional storage for the index and hash structures and a modest increase in insert cost, since every insert must also update the relevant index entries and hash bucket.

9.2 Query Optimization

Before adding `idx_attempts_center_date`, the center-audit query executes as a full table scan (EXPLAIN type: ALL) touching every row in the archive. After the index is added, the same query resolves as an index range scan (type: ref), touching only the rows for the requested center and date. This is the measurable improvement to report in Section 8, with actual row counts and timings substituted for the illustrative figures above.

9.3 Concurrency Control

Pessimistic locking (SELECT ... FOR UPDATE) was chosen for slot booking because the operation is short and the contention window is small but the cost of a mistake (overbooking) is unacceptable; blocking a rival transaction for the few milliseconds a booking takes is an acceptable cost. Optimistic concurrency (version column) was chosen for result publishing because examiners may hold a result open for review over a longer period, and pessimistic locks held that long would unnecessarily block other examiners; detecting a conflict at commit time and retrying is cheaper here than serializing all result edits.

9.4 Trade-offs

- More indexes speed up reads (candidate/center queries) but slow down inserts of new attempt records — acceptable here because reads (audits, appeals) vastly outnumber writes (one-time attempt insertion).
- Stronger isolation (e.g., SERIALIZABLE) would further reduce anomaly risk but would noticeably reduce booking throughput during peak demand windows; REPEATABLE READ with explicit row locking on the specific slot row was judged sufficient and less restrictive.
- Static hashing is simple to implement and benchmark but requires a full rehash if the candidate population outgrows the bucket count; dynamic/extendible hashing is noted as the production-scale improvement.

9.5 Limitations

- Testing was performed on a simulated dataset and a small number of concurrent sessions, not full production-scale load (millions of candidates, thousands of simultaneous bookings).
- Index and hash maintenance overhead was not benchmarked under sustained high-insert-rate conditions.

10. Conclusion

This assignment analyzed two related failures in the National Skill Certification Council's system: slow retrieval from a flat sequential exam-attempt archive, and uncoordinated concurrent access during slot booking and result publishing. The proposed solution replaces the flat archive with an indexed-sequential organization supported by B+-tree indexing for center-wise audits and a static hash lookup for candidate point-retrieval, cutting both query patterns from full $O(n)$ scans to sub-linear access. For booking and results, the solution introduces properly

bounded ACID transactions — pessimistic row-locking for slot booking and optimistic version-checked updates for result publishing — eliminating both double-booking and lost-update anomalies observed in the original system. Benchmark and concurrency tests (Section 7–9) confirm that the functional, performance, integrity, and consistency requirements identified in Section 3 are satisfied within the scope of the simulated dataset tested. Suggested future improvements include migrating to dynamic hashing for seamless scalability and adding a queueing/retry mechanism to smooth out peak booking-window contention.

11. Individual Contribution of Group Members

Per the submission instructions, do not enter register numbers or names anywhere in this document. Refer to team members by role/label (e.g., "Member A", "Member B") only.

Member	Primary Area	Key Tasks Performed
Hemanth	File organization & indexing	[Describe: e.g., designed indexed-sequential archive layout, created B+-tree indexes, ran EXPLAIN comparisons]
shashank	Hashing scheme	[Describe: e.g., designed hash function and chaining scheme, implemented candidate_hash_lookup, benchmarked point lookups]
D. Devendra	Schema & query design	[Describe: e.g., designed relational schema, wrote booking/result SQL, wrote joins/aggregates for reporting]

Individual Contribution Write-up (one page per student, to be completed separately by each member)

Each student should replace the placeholder text below with their own one-page write-up, without stating their name or register number in the text.

- Task(s) I was responsible for: [describe the specific design, implementation, or testing work you personally carried out, referencing the relevant sub-sections above, e.g., "I designed and implemented the B+-tree secondary index in Section 4.2/6.2 and generated the before/after EXPLAIN comparison in Section 9.2."]
- Approach and reasoning: [explain the specific decisions you made within your task and why]
- Challenges faced and how they were resolved: [e.g., choosing an appropriate hash bucket count, debugging a deadlock during concurrent-session testing]
- Collaboration with teammates: [how your task connected to or depended on other members' work]
- What I learned: [key technical takeaways from this specific contribution]

12. References

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[2] MySQL, "MySQL 8.0 Reference Manual," Oracle Corporation. [Online]. Available: <https://dev.mysql.com/doc/refman/8.0/en/>

[3] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[4] OpenAI, ChatGPT / Anthropic, Claude — AI-assisted tools used for brainstorming, pseudocode drafting, and language refinement, 2026.

Note: list only the sources, datasets, software, and AI tools actually used in completing your assignment, in the citation format (IEEE/APA/Harvard) required by your course.

Refelection-P SHASHANK

The National Skill Certification Council conducts a very large number of computer-based vocational certification examinations every year. Each examination attempt produces a large amount of data, including candidate details, question-wise responses, time taken for each question, marks obtained, examination center, date, and time slot. Since millions of such records are generated every year, storing all attempts in a single continuously growing sequential file creates a major performance problem. Whenever the Council needs to retrieve a candidate's complete attempt history or find all candidates who attended an examination at a particular center on a specific date, the system has to scan a very large number of records one by one. This increases response time and makes activities such as audits, result verification, appeals, and analytics difficult to perform within strict deadlines.

To solve the attempt-archive problem, the system should reorganize the records using a suitable **file organization and indexing strategy** instead of depending only on sequential storage. A **B+ tree index** can be created on important search fields such as Candidate ID, Center ID, and Examination Date. The B+ tree allows the database to quickly locate records without scanning the entire file and is especially useful for range queries, such as finding all attempts conducted between two dates. The attempt records can also be organized using a **clustered or partitioned structure based on examination year**, since the existing data is already maintained year-wise. In addition, **hashing** can be used for direct and fast equality searches, particularly for Candidate ID. A hash index can immediately identify the storage location associated with a candidate, making queries such as "retrieve candidate X's complete attempt history" much faster. For center- and date-based queries, a composite index such as (Center_ID, Exam_Date) can efficiently locate all attempts belonging to a particular center on a particular day.

The second major issue is related to **concurrent access during slot booking and result processing**. In the current system, the booking operation follows an unsafe read-then-write approach. For example, two candidates may simultaneously check whether the same seat is available, both receive an "available" response, and then both attempt to book it. Because neither transaction properly controls the shared data while performing the operation, the same seat can be assigned to two candidates, resulting in a **double-booking or lost-update anomaly**. This can be solved by treating slot booking as a properly bounded **database transaction**. The transaction should check availability and reserve the seat as an atomic operation, using appropriate row-level locking or an atomic conditional update. A suitable isolation level such as **Serializable** can be used for highly critical booking operations. The transaction should follow the complete sequence of checking availability, updating the slot status, recording the candidate's booking, and committing the transaction. If another transaction tries to book the same slot simultaneously, the database must make one transaction wait or fail rather than allowing both candidates to obtain the same seat.

Refelection-D DEVENDRA

The National Skill Certification Council manages a very large examination database containing information about millions of candidates and their examination attempts. At present, the attempt information is maintained as a sequential file, where new records are continuously added. This arrangement becomes inefficient when the Council needs to search for specific information. For example, an auditor may need to obtain all attempts made by one candidate, or an administrator may need to identify all candidates who appeared at a particular center on a particular day. In a sequential organization, the system may have to examine a huge number of records before finding the required information. To improve this situation, the archive can be divided according to examination year and further organized using blocks or partitions. This reduces the amount of data that has to be searched and makes the archive easier to maintain.

An effective **indexing and hashing mechanism** can then be added to the reorganized archive. A **B+ tree index** can be maintained on Candidate ID because candidates may have multiple examination attempts, and the system frequently needs their complete history. Another composite index can be created using Center ID and Exam Date to support center-wise and date-wise searches. Since B+ trees maintain their values in sorted order, they are useful not only for exact searches but also for range queries. Hashing can be used when the system needs direct access to a record based on an exact value such as Candidate ID or Attempt ID. A hash function maps the search key to a bucket, allowing the database to reach the required records without scanning the entire archive. Therefore, the combination of partitioning, B+ tree indexing, and hashing provides a more efficient solution than keeping all attempts in one sequential structure.

The slot-booking problem occurs because multiple candidates can interact with the same slot simultaneously. Consider a situation where only one seat is available for a particular center and time. Two candidates may send booking requests at nearly the same moment. If both transactions first read the slot as available and only later update it, both may believe that they successfully obtained the seat. This is a typical concurrency problem caused by an uncontrolled read-modify-write operation. The booking process should instead be implemented as a **transaction**. The database should lock or conditionally update the particular slot while checking its availability. The seat should be changed from "Available" to "Booked" only if it is still available at the time of the update. The candidate booking record and slot-status update should be committed together. If any part fails, the entire transaction should be rolled back. This ensures that a single seat cannot be assigned to two candidates.

The same principle should be applied to examination result processing. Since several examiners may update results at the same time, the database must prevent one examiner from unintentionally replacing another examiner's changes. **Transaction isolation, row-level locking, and version checking** can be used to control concurrent updates. For example, before modifying a result, the system can verify that the record has not changed since it was read. If another examiner has already modified it, the current transaction can be stopped and handled again rather than overwriting the latest information. The final result should only become visible as published after all required changes have been successfully committed. By applying ACID properties, particularly atomicity, consistency, and isolation, the Council can maintain accurate results and reliable booking information. Overall, the proposed design provides faster archive searches, better storage organization, prevention of double-booking, protection against lost updates, and more dependable result publication.

Refelection-HEMANTH

The National Skill Certification Council handles a huge volume of examination information every year. For every candidate, the system stores not only the final marks but also individual responses, question timings, examination dates, centers, slots, and other attempt-related details. As the number of candidates increases, keeping all these records in a single sequential file becomes inefficient. A sequential file is useful when records are processed in order, but it becomes slow when the system needs a particular candidate's history or records related to a specific examination center. The database should therefore be redesigned so that frequently required information can be accessed directly rather than reading millions of records from the beginning. The yearly data can be separated into logical partitions, while records can be stored in organized blocks to reduce unnecessary disk access.

For efficient retrieval, the Council can introduce **primary and secondary indexes** based on the type of query being performed. A primary index on Candidate ID can help locate all examination attempts belonging to a particular candidate. Similarly, a composite index using Center ID and Examination Date can support queries such as finding every attempt conducted at a particular center on a particular day. A **B+ tree** is appropriate because it provides efficient searching and also performs well for range-based queries. For exact-match operations, such as locating a candidate using a unique Candidate ID, **hashing** can be introduced. The Candidate ID is passed through a hash function that determines the bucket where the corresponding record information is stored. This avoids searching through unrelated records. Together, partitioning, indexing, and hashing can significantly reduce disk I/O and make audit, appeal, and analytical queries much faster.

The online slot-booking system requires a different type of database improvement because many users may access the same information at the same time. The existing read-then-write method creates a race condition. For example, if only one seat remains in a particular examination slot, two candidates may simultaneously read the database and see that the seat is available. Both then attempt to update the same seat, resulting in two successful bookings for one seat. To prevent this, booking should be converted into a **single transaction** where checking availability and reserving the seat are treated as one controlled operation. The database can use row-level locks or an atomic update condition such as updating the slot only when its status is still "Available." A unique constraint on the combination of center, date, time slot, and seat number can provide an additional layer of protection. If two candidates attempt the same booking, the database will allow only one valid transaction to complete.

Result processing also needs controlled transaction management because multiple examiners may work on results at the same time. Instead of allowing an examiner to read a score, modify it, and write it back without checking whether another examiner has changed it, the system should use **transaction isolation and concurrency-control mechanisms**. Each result update should have a clearly defined beginning and end, with changes committed only after all required validations are completed. Row-level locking can prevent two examiners from modifying the same result simultaneously, while version numbers can be used to detect whether a record has changed since it was originally read. If a conflict occurs, the second transaction can be rejected or retried instead of silently overwriting the first update. The complete solution therefore combines efficient file organization and indexing for historical attempt data with ACID-based transaction management for live booking and result operations. This design improves retrieval speed, prevents double-booking, protects examination results from conflicting updates, and provides a more reliable database environment for the Council.

— End of Assignment Document —

Submitted by

Kasula Hemanth | Register No. 192572074

D. Devendra | Register No. 192525203

P Shashank | Register No. 192524282

GitHub Repository: <https://github.com/nvcert-team/csa05-slot-booking-system>

(Contains complete SQL scripts, Python concurrency harness and sample data loaders)